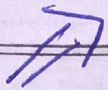


Output from LLM is a fixed, predictable, machine readable format (like JSON, dictionary, schema) Date



Structured Output :-

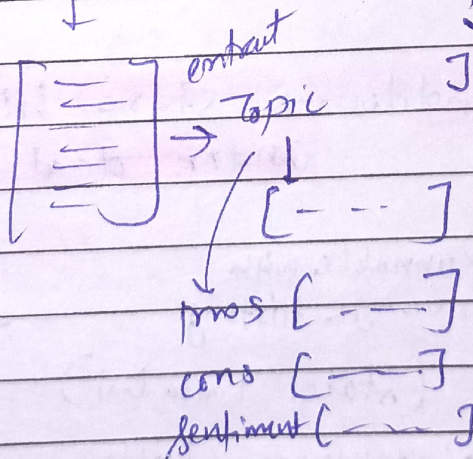
- In langchain, structured output refers to the practice of leaving language models return responses in a well-defined data format (for example JSON) rather than free form text. This makes the model output easier to parse and works with programmatically.

Use cases :-

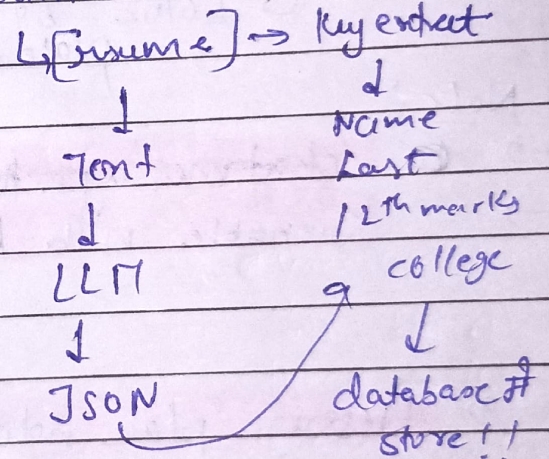
- Data Extraction
- API building
- Agents

Chatbots - Tools like calculator

Amazon
↓
products
↓
review

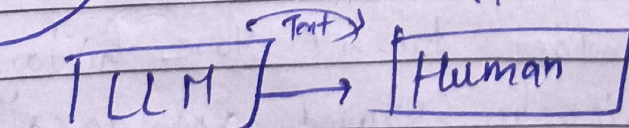


Naukri.com



API

Structured output → powerful Technique hai !!



no integration like Database, APIs with llm

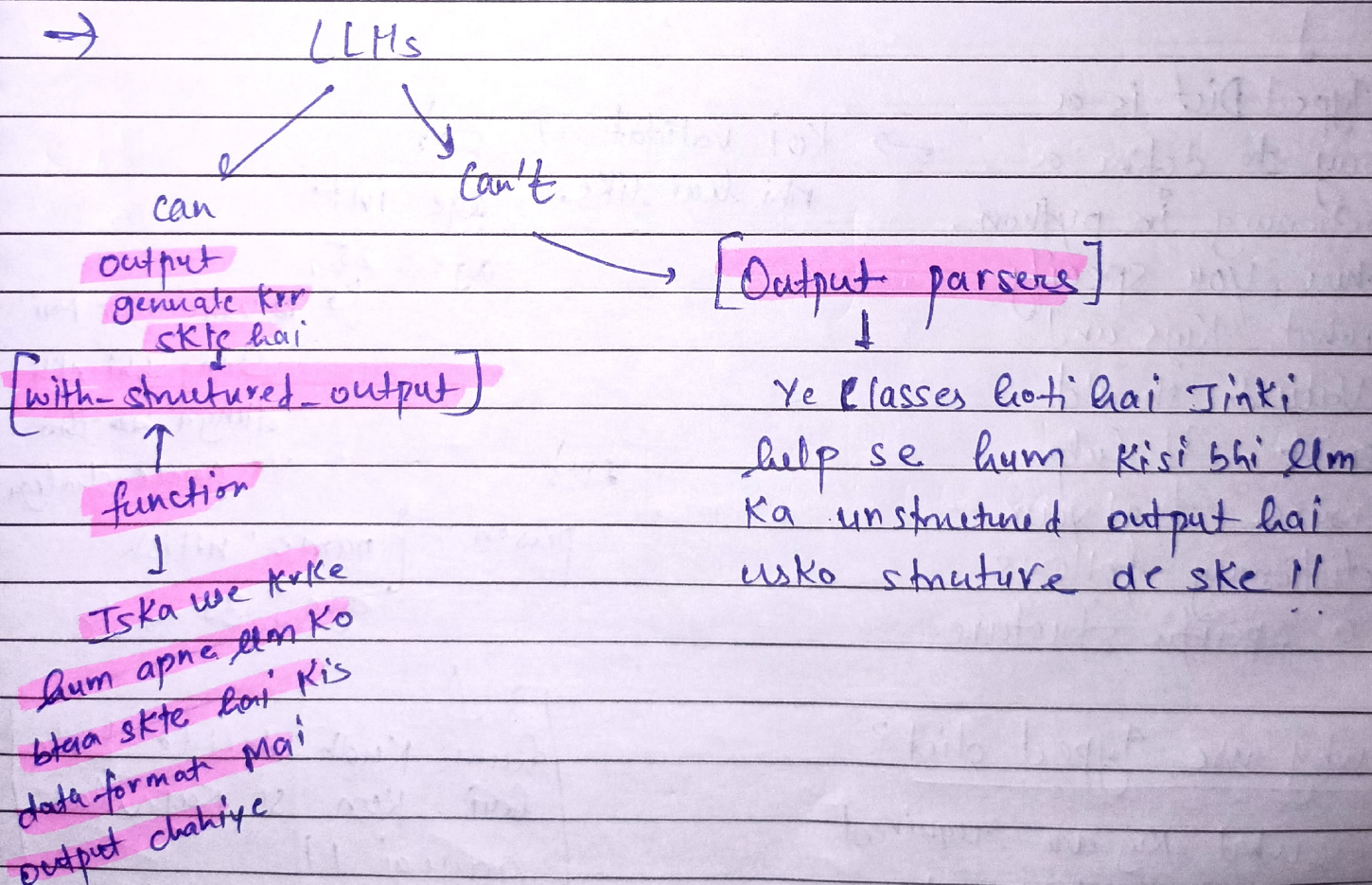
Teacher's Signature

~~LLMs~~ → why do we need structured output??

• Because in such AI Apps

1. chatbots needs field.
2. Agents needs actions
3. API need JSON
4. Database need structured data
5. Automation needs predictable Keys.

without structure → app breaks.

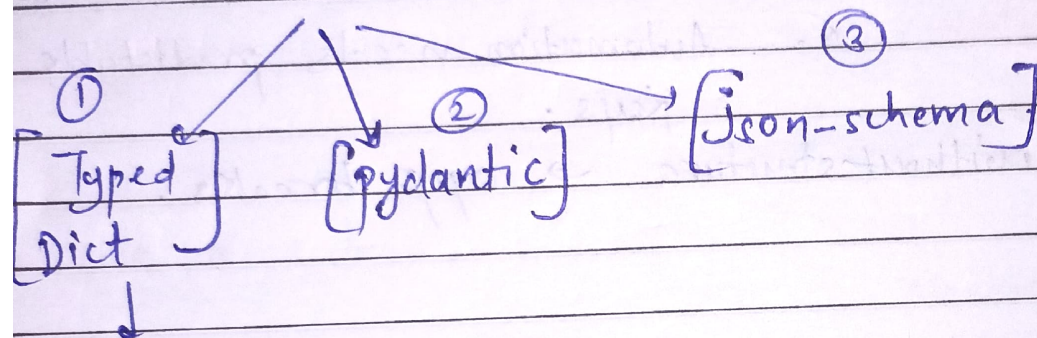


①

→ With structured output

• (model - invoke)

↳ Just iske pehle with-structured-output ko call krna padta hai aur yaha pe data format batana hota hai !!



Typed Dict is a way to define a dictionary in python where you specify what keys and values should exist. it help ensure that your dictionary follows a specific structure

→ Koi validation nhi hai like
 age int
 age = 25;

↳ yaha agar Mai String bhi rkhdunga to ye code chalga.

ent;

person = { name: nitish
 age: 35 → str

• why use Typed dict?

1. what key are required and what types of values they ~~are~~ should have.

2. It doesn't validate data at runtime.

↓
 hum kadh decide krte hai kon si key value gayegi !!
 way to define a dictionary !!

TypeDict_demo.py → simple, fast, strict validation Kam

from typing import TypedDict

class Person (TypedDict):

name : str

age : int

new_person : person = {'name' : 'nitish', 'age' : '35'}

print (new_person)

Output :-

{ 'name' : 'nitish', 'age' : '35' }

pydantic model (most common) → sentiment → string, confidence → float

```
from pydantic import BaseModel

class Sentiment (BaseModel):
    sentiment : str
    confidence : float
```

- automatic validation
- clean python object output.
- Best for production python apps.

Ab hm ko bola jata hai output isi format mai de.

JSON schema :- → universal structured format

→ strict validation ke liye

②

Date

→ pydantic → is a data validation and data parsing library for python. it ensures that the data you work with is correct, structured and type safe.

→ Uses :-

- Default values
- optional fields
- Coerce
- Builtin validation.
- field function → default values, constraints, description, regex expressions.
- Return pydantic object → convert to json / dict.

-----x-----x-----x-----x-----

ex:- from pydantic import BaseModel

class Student (BaseModel):

name : str

new_student = { 'name' : 'nitish' }

student = Student (** new_student)

print (type(student))

Output :-

name = 'nitish'

→ new_student = { 'name' : 32 }

↳ it gives error,
error Input should be a valid string.

③

Date _____

→ JSON Schema :

→ machine-readable rules define Karta hai

→ Use when : APIs / cross platform system bana rhe ho

```
{  
  "name" : "string",  
  "score" : "number"  
}
```

→ language independent

→ API friendly

→ strict structure.

-
- pydantic → python Validation King
 - Typed Dict → light weight typing
 - JSON Schema → universal format.
-

Interview :

"Langchain structured Output supports schemas like pydantic for validation, Typed Dict for light weight typing and JSON schema for universal structured responses."

Teacher's Signature

① Use Typed Dict if :

- you only need type hints
- you don't need validation
- you trust the LLM to return correct data.

② Use Pydantic if :

- you need data validation (eg: Sentiment must be
- you need default values if "positive", "neutral" or "negative"
- you want the LLM misses fields -)
- automatic Type conversion (e.g $\rightarrow$ "100" $\leftrightarrow$ 100)

③ Use JSON Schema if :

- you don't want to import extra python libraries
- you need validation but don't need python objects.
- you want to define structure in a standard JSON format.

→ When to Use what??

Feature	Typed dict	pydantic	JSON schema
Basic Structure	✓	✓	✓
Type enforcement	✓	✓	✓
Data validation	X	✓	✓
Default values	X	✓	X
Automatic conversion	X	✓	X
Cross language compatibility	X	X	✓

Teacher's Signature